Method:

In this project, we studied different neural network architectures for decoding text from EMG signals using CTC. The goal was to understand how different sequence models perform and whether combining convolutional and recurrent models improves performance.

All models used EMG signals as input with 528 features. The decoding method was CTC greedy decoding across all experiments to ensure fair comparison between models.

Data Preprocessing:

The raw EMG signals were transformed using a log spectrogram representation. This converts time-domain EMG signals into a time–frequency representation that is easier for neural networks to learn from.

During training, several data augmentation techniques were applied:

- Conversion to tensor format
- - Random band rotation to simulate sensor variation
- Temporal alignment jitter to simulate timing differences
- Gaussian noise injection to improve robustness
- Log spectrogram transformation
- SpecAugment masking in time and frequency
- Feature normalization

Validation and test data used only deterministic transformations (log spectrogram and normalization) to avoid randomness during evaluation. These preprocessing steps were designed to improve generalization and reduce overfitting.

Model Architectures:

We experimented with multiple architectures to compare different ways of modeling temporal EMG signals.

- CNN (Baseline): The CNN model used stacked temporal convolution blocks to extract local patterns from EMG signals. Convolutions capture short-term temporal dependencies and are efficient to train. This model served as the baseline for comparison.
- LSTM: The LSTM model processes sequences using gated recurrent units that maintain memory over long time periods. This allows the model to capture temporal dependencies across long EMG sequences.
- Vanilla RNN: A standard recurrent neural network was also tested. Unlike LSTM, it does not contain gating mechanisms. This experiment was included to study how important

gating is for stable sequence learning. Gradient clipping and regularization techniques were later added to stabilize training.

- GRU: GRU models were evaluated as a simpler gated alternative to LSTM. GRUs use fewer parameters while still maintaining temporal memory.
- Hybrid Architectures (CNN + RNN variants): Hybrid models combined convolutional layers with recurrent layers: CNN extracts local temporal features, recurrent layers model long-range dependencies. We evaluated CNN + LSTM, CNN + RNN, and CNN + GRU. The intuition is that CNN layers learn low-level signal patterns while recurrent layers learn sequence structure.

Results:

The baseline CNN model achieved a test Character Error Rate (CER) of 24.96 without transformations. Adding Gaussian noise and normalization significantly improved performance, reducing test CER to 20.83.

The LSTM model performed better than the baseline when properly tuned. A deeper LSTM achieved a test CER of 20.83, similar to the improved CNN model. However, when heavy augmentations were applied, LSTM performance degraded and test CER increased to 34.06, suggesting sensitivity to preprocessing choices.

The CNN + LSTM hybrid produced the best performance overall. After tuning dropout, learning rate, and convolution channels, the model achieved a test CER of 18.0030, which is a large improvement over the CNN baseline.

Vanilla RNN models performed poorly initially, with test CER above 60. After applying gradient clipping and regularization, performance improved to 28.29, but still remained worse than LSTM-based models.

CNN + RNN hybrids improved over standalone RNNs, reaching a best test CER of 23.43, showing that convolutional feature extraction helps stabilize weaker recurrent models.

GRU models performed unexpectedly poorly. Most GRU configurations produced very high CER values, often above 80 or even failing completely with CER near 100. CNN + GRU hybrids also failed to converge effectively.

Overall, the CNN + LSTM hybrid achieved the best performance among all tested architectures.

Discussion:

Convolutional layers were highly effective at extracting useful patterns from EMG signals. Even simple CNN models performed well, and preprocessing improvements further reduced errors, showing that local temporal structure is important.

Gated models like LSTM were necessary for stable sequence learning because vanilla RNNs struggled with vanishing and exploding gradients. This explains why modern sequence models depend on gating mechanisms.

The best results came from combining CNN and LSTM. CNN layers learned short-term features, while LSTM layers captured long-term dependencies, allowing the hybrid model to achieve the lowest CER.

Preprocessing strongly influenced model performance. While data augmentation helped CNNs, it sometimes hurt LSTM and GRU models, suggesting recurrent models are more sensitive to noisy data.

GRUs performed worse than expected, likely because they have fewer gates than LSTMs, limiting their ability to model complex temporal patterns. Training instability in CNN + GRU models supports this idea.

Overall, careful architecture design and tuning greatly improved results, with the final CNN + LSTM model lowering test CER from 24.96 to 18.00.

Models:

Common across all:

- Decoder = CTC greedy
- Gaussian Noise STD is 0.01 unless otherwise stated

1. CNN (Baseline)
    - Batch Size = 32
    - Epochs = 40
    - Adam Learning Rate = 1e-3
    - No transformation
    - In features = 528
    - MLP features = [384]
    - Block channels = [24, 24, 24, 24]
    - Kernel Width = 32
    - Window Length = 8000
    - Padding = [1000, 200]
    - Validation CER = 23.9698
    - Test CER = 24.9622

2. CNN
    - Batch Size = 16
    - Epochs = 25
    - Num Workers = 8
    - No transformation
    - In features = 528
    - MLP features = [384]
    - Block channels = [24, 24, 24, 24]
    - Kernel Width = 32
    - Window Length = 8000
    - Padding = [1000, 200]
    - Validation CER = 24.2357
    - Test CER = 24.3138

3. CNN - Gaussian Noise
    - Batch Size = 16
    - Epochs = 30
    - Num Workers = 8
    - Transformations:

        - Train: To Tensor → Band Rotation → Temporal jitters → Gaussian Noise → Logspec → Specaug
        - Test: To Tensor → Logspec
    - In features = 528
    - MLP features = [384]
    - Block channels = [24, 24, 24, 24]
    - Kernel Width = 32
    - Window Length = 8000
    - Padding = [1000, 200]
    - Validation CER = 21.2007
    - Test CER = 22.0661

4. CNN - Gaussian Noise & Normalization
    - Batch Size = 16
    - Epochs = 30
    - Num Workers = 8
    - Transformations:
        - Train: To Tensor → Band Rotation → Temporal jitters → Gaussian Noise → Logspec → Normalization → Specaug
        - Test: To Tensor → Logspec → Normalization
    - In features = 528
    - MLP features = [384]
    - Block channels = [24, 24, 24, 24]
    - Kernel Width = 32
    - Window Length = 8000
    - Padding = [1000, 200]
    - Validation CER = 19.0961
    - Test CER = 20.8342

5. LSTM
    - Dropout = 0.3
    - Batch Size = 16
    - Epochs = 30
    - In features = 528
    - MLP features = [256]
    - LSTM Hidden = 192
    - LSTM Layers = 2
    - No transformations
    - Val CER = 24.7566
    - Test CER = 23.7303

6. LSTM
    - Dropout = 0.3
    - Batch Size = 16
    - Epochs = 35
    - In features = 528
    - MLP features = [256, 128]
    - LSTM Hidden = 256
    - LSTM Layers = 3
    - No transformations
    - Val CER = 21.9248
    - Test CER = 20.8342

7. LSTM - Gaussian Noise & Normalization
    - Dropout = 0.3
    - Batch Size = 16
    - Epochs = 30
    - In features = 528
    - MLP features = [256, 128]
    - LSTM Hidden = 256
    - LSTM Layers = 3
    - Transformations:
        - Train: To Tensor → Band Rotation → Temporal jitters → Gaussian Noise → Normalization → Logspec
        - Test: To Tensor → Normalization → Logspec
    - Val CER = 33.4292
    - Test CER = 34.0609

8. CNN + LSTM Hybrid - Gaussian Noise & Normalization
    - Dropout = 0.3
    - Batch Size = 16
    - Epochs = 30
    - Adam Learning Rate = 5e-4
    - Adam Weight Decay = 1e-5
    - In features = 528
    - MLP features = [256, 128]
    - Block Channels = [128, 128, 128]
    - Kernel Width = 5
    - LSTM Hidden = 256
    - LSTM Layers = 3

- Transformations:
    - Train: To Tensor → Band Rotation → Temporal jitters → Gaussian Noise → Logspec → Normalize → Specaug
    - Test: To Tensor → Logspec → Normalize
- Val CER = 20.4646
- Test CER = 19.8617

9. CNN + LSTM Hybrid - Gaussian Noise & Normalization
    - Dropout = 0.3
    - Batch Size = 24
    - Epochs = 45
    - Adam Learning Rate = 2e-4
    - Adam Weight Decay = 1e-2
    - Log Specaug N Time Mask = 2
    - Log Specaug Time Mask Param = 15
    - Log Specaug Freq Mask Param = 3
    - In features = 528
    - MLP features = [384, 192]
    - Block Channels = [24, 24, 24, 24]
    - Kernel Width = 7
    - LSTM Hidden = 384
    - LSTM Layers = 4
    - Transformations:
        - Train: To Tensor → Band Rotation → Temporal jitters → Gaussian Noise → Logspec → Normalize → Specaug
        - Test: To Tensor → Logspec → Normalize
    - Val CER = 29.2920
    - Test CER = 26.9289

10. CNN + LSTM Hybrid - Gaussian Noise & Normalization
    - Dropout = 0.1
    - Batch Size = 16
    - Epochs = 60
    - Adam Learning Rate = 3e-4
    - Adam Weight Decay = 3e-5
    - In features = 528
    - MLP features = [256, 128]
    - Block Channels = [64, 128, 128, 64]
    - Kernel Width = 5
    - LSTM Hidden = 320

- LSTM Layers = 3
- Transformations:
    - Train: To Tensor → Band Rotation → Temporal jitters → Gaussian Noise → Logspec → Normalize → Specaug
    - Test: To Tensor → Logspec → Normalize
- Val CER = 18.5177
- Test CER = 18.0030

11. RNN
    - Batch Size = 16
    - Epochs = 25
    - Non Linearity = Tanh
    - No Transformations
    - In features = 528
    - MLP features = [256, 128]
    - RNN Hidden = 256
    - RNN Layers = 3
    - Val CER = 71.4602
    - Test CER = 64.0372

12. RNN - Gaussian Noise & Normalization
    - Batch Size = 24
    - Epochs = 45
    - Adam Learning Rate = 0.0007
    - Adam Weight Decay = 0.01
    - Gaussian Noise STD = 0.015
    - Non Linearity = Tanh
    - Transformations:
        - Train: To Tensor → Band Rotation → Temporal jitters → Gaussian Noise → Logspec → Normalize → Specaug
        - Test: To Tensor → Logspec → Normalize
    - In features = 528
    - MLP features = [256, 128]
    - RNN Hidden = 320
    - RNN Layers = 4
    - Val CER = 99.9115
    - Test CER = 100.0

13. RNN - Gaussian Noise & Normalization
    - Batch Size = 16

- Epochs = 35
- Adam Learning Rate = 0.0003
- Adam Weight Decay = 0.01
- Gaussian Noise STD = 0.01
- Non Linearity = ReLU
- Transformations:
    - Train: To Tensor → Band Rotation → Temporal jitters → Gaussian Noise → Logspec → Normalize → Specaug
    - Test: To Tensor → Logspec → Normalize
- In features = 528
- MLP features = [256, 128]
- RNN Hidden = 192
- RNN Layers = 2
- Val CER = 44.3584
- Test CER = 43.8945

14. RNN - Gaussian Noise & Normalization, Vanilla RNN, Gradient Clipping, Pascanu's Regularization
    - Batch Size = 16
    - Epochs = 30
    - Gradient Clip = 1.0
    - Gradient Clip Algorithm = Norm
    - Adam Learning Rate = 0.0002
    - Adam Eps = 1e-8
    - Adam Weight Decay = 0.0001
    - Gaussian Noise STD = 0.02
    - Non Linearity = ReLU
    - Transformations:
        - Train: To Tensor → Band Rotation → Temporal jitters → Gaussian Noise → Logspec → Specaug → Normalize
        - Test: To Tensor → Logspec → Normalize
    - In features = 528
    - MLP features = [256, 128]
    - RNN Hidden = 256
    - RNN Layers = 3
    - Val CER = 32.8097
    - Test CER = 31.7269

15. RNN - Gaussian Noise & Normalization, Vanilla RNN, Gradient Clipping, Pascanu's Regularization

- Dropout = 0.2
- Batch Size = 16
- Epochs = 40
- Gradient Clip = 1.0
- Gradient Clip Algorithm = Norm
- Adam Learning Rate = 0.0002
- Adam Eps = 1e-8
- Adam Weight Decay = 0.0001
- Gaussian Noise STD = 0.02
- Non Linearity = ReLU
- Transformations:
    - Train: To Tensor → Band Rotation → Temporal jitters → Gaussian Noise → Logspec → Specaug → Normalize
    - Test: To Tensor → Logspec → Normalize
- In features = 528
- MLP features = [256, 128]
- RNN Hidden = 320
- RNN Layers = 3
- Val CER = 30.0885
- Test CER = 28.2905

16. CNN + RNN Hybrid, Vanilla RNN, Gradient Clipping, Pascanu's Regularization
    - Batch Size = 16
    - Epochs = 40
    - Gradient Clip = 1.0
    - Gradient Clip Algorithm = Norm
    - Adam Learning Rate = 0.0002
    - Adam Eps = 1e-8
    - Adam Weight Decay = 0.0001
    - Gaussian Noise STD = 0.02
    - Non Linearity = Tanh
    - Transformations:
        - Train: To Tensor → Band Rotation → Temporal jitters → Gaussian Noise → Logspec → Specaug → Normalize
        - Test: To Tensor → Logspec → Normalize
    - In features = 528
    - MLP features = [256, 128]
    - Block Channels = [128, 128, 128]
    - Kernel Width = 5
    - RNN Hidden = 256

- RNN Layers = 3
- Val CER = 27.5
- Test CER = 27.4692

17. CNN + RNN Hybrid, Vanilla RNN, Gradient Clipping, Pascanu's Regularization
    - Dropout = 0.2
    - Batch Size = 16
    - Epochs = 60
    - Gradient Clip = 1.0
    - Gradient Clip Algorithm = Norm
    - Adam Learning Rate = 0.0002
    - Adam Eps = 1e-8
    - Adam Weight Decay = 0.0001
    - Gaussian Noise STD = 0.015
    - Non Linearity = ReLU
    - Log Specaug N Time Mask = 2
    - Log Specaug Time Mask Param = 15
    - Log Specaug Freq Mask Param = 3
    - Transformations:
        - Train: To Tensor → Band Rotation → Temporal jitters → Gaussian Noise → Logspec → Normalize → Specaug
        - Test: To Tensor → Logspec → Normalize
    - In features = 528
    - MLP features = [256, 128]
    - Block Channels = [128, 128, 128]
    - Kernel Width = 5
    - RNN Hidden = 256
    - RNN Layers = 3
    - Val CER = 22.6106
    - Test CER = 23.4277

18. GRU - Gaussian Noise & Normalization
    - Batch Size = 16
    - Epochs = 45
    - Gradient Clip = 1.0
    - Gaussian Noise STD = 0.00
    - Adam LR = 1e-4
    - Adam Weight Decay = 0.001
    - Dropout = 0.1
    - No Transformations

- In features = 528
- MLP features = [256]
- GRU Hidden = 256
- GRU Layers = 2
- Val CER = 86.7478
- Test CER = 81.3486

19. GRU - Gaussian Noise & Normalization
    - Batch Size = 24
    - Epochs = 60
    - Gradient Clip = 1.0
    - Gaussian Noise STD = 0.02
    - Adam LR = 3e-4
    - Adam Weight Decay = 1e-2
    - Dropout = 0.3
    - Transformations:
        - Train: To Tensor → Band Rotation → Temporal jitters → Gaussian Noise → Logspec → Normalize → Specaug
        - Test: To Tensor → Logspec → Normalize
    - In features = 528
    - MLP features = [256, 128]
    - GRU Hidden = 320
    - GRU Layers = 3
    - Val CER = 99.9779
    - Test CER = 99.9352

20. GRU - Gaussian Noise & Normalization
    - Batch Size = 16
    - Epochs = 60
    - Gradient Clip = 0.5
    - Gaussian Noise STD = 0.00
    - Adam LR = 1e-4
    - Adam Weight Decay = 0.001
    - Dropout = 0.1
    - Transformations:
        - Train: To Tensor → Band Rotation → Temporal jitters → Gaussian Noise → Logspec → Normalize → Specaug
        - Test: To Tensor → Logspec → Normalize
    - In features = 528
    - MLP features = [256]

- GRU Hidden = 256
- GRU Layers = 2
- Val CER = 89.6018
- Test CER = 85.7359

21. CNN + GRU Hybrid, Gaussian Noise & Normalization
    - Batch Size = 24
    - Epochs = 18
    - Gradient Clip = 1.0
    - Gradient Clip Algorithm = Norm
    - Adam Learning Rate = 0.0008
    - Adam Weight Decay = 0.01
    - Gaussian Noise STD = 0.015
    - Dropout = 0.3
    - CNN Dropout = 0.1
    - Transformations:
        - Train: To Tensor → Band Rotation → Temporal jitters → Gaussian Noise → Logspec → Normalize → Specaug
        - Test: To Tensor → Logspec → Normalize
    - In Features = 528
    - MLP Features = [256, 128]
    - Block Channels = [64, 128, 128, 64]
    - Kernel Width = 5
    - GRU Hidden = 320
    - GRU Layers = 3
    - Val CER = 100.0
    - Test CER = 100.0

22. CNN + GRU Hybrid, Gaussian Noise & Normalization
    - Batch Size = 16
    - Epochs = 22
    - Gradient Clip = 0.5
    - Gradient Clip Algorithm = Norm
    - Adam Learning Rate = 0.0003
    - Adam Weight Decay = 0.01
    - Gaussian Noise STD = 0.005
    - Dropout = 0.3
    - CNN Dropout = 0.1
    - Transformations:

    - Train: To Tensor → Band Rotation → Temporal jitters → Gaussian Noise → Logspec → Normalize → Specaug
    - Test: To Tensor → Logspec → Normalize
- In Features = 528
- MLP Features = [256, 128]
- Block Channels = [64, 96, 96, 64]
- Kernel Width = 5
- GRU Hidden = 256
- GRU Layers = 2
- Val CER = 100.0
- Test CER = 100.0